

Lecture 10: Backpropagation

Instructor: Stephanie Schoch

CSCI 680: Natural Language Processing
Department of Computer Science, College of William & Mary
September 25, 2026



Lecture Outline

- <https://forms.gle/ts1b5BXAXnNDMfJN7>



- Feed-Forward Networks: Language Modeling
- Backpropagation
- RNNs

FFNN: Language Modeling

Neural Language Models

Language Modeling: Calculating the probability of the next word in a sequence given some history.

- Neural network LMs far outperform n-gram language models

Feedforward Neural Language Models

Task: predict next word w_t

given prior words $w_{t-1}, w_{t-2}, w_{t-3}, \dots$

Problem: Now we're dealing with sequences of arbitrary length.

Solution: Sliding windows (of fixed length)

$$P(w_t | w_1, \dots, w_{t-1}) \approx P(w_t | w_{t-N+1}, \dots, w_{t-1})$$

Inference in a Feedforward Language Model

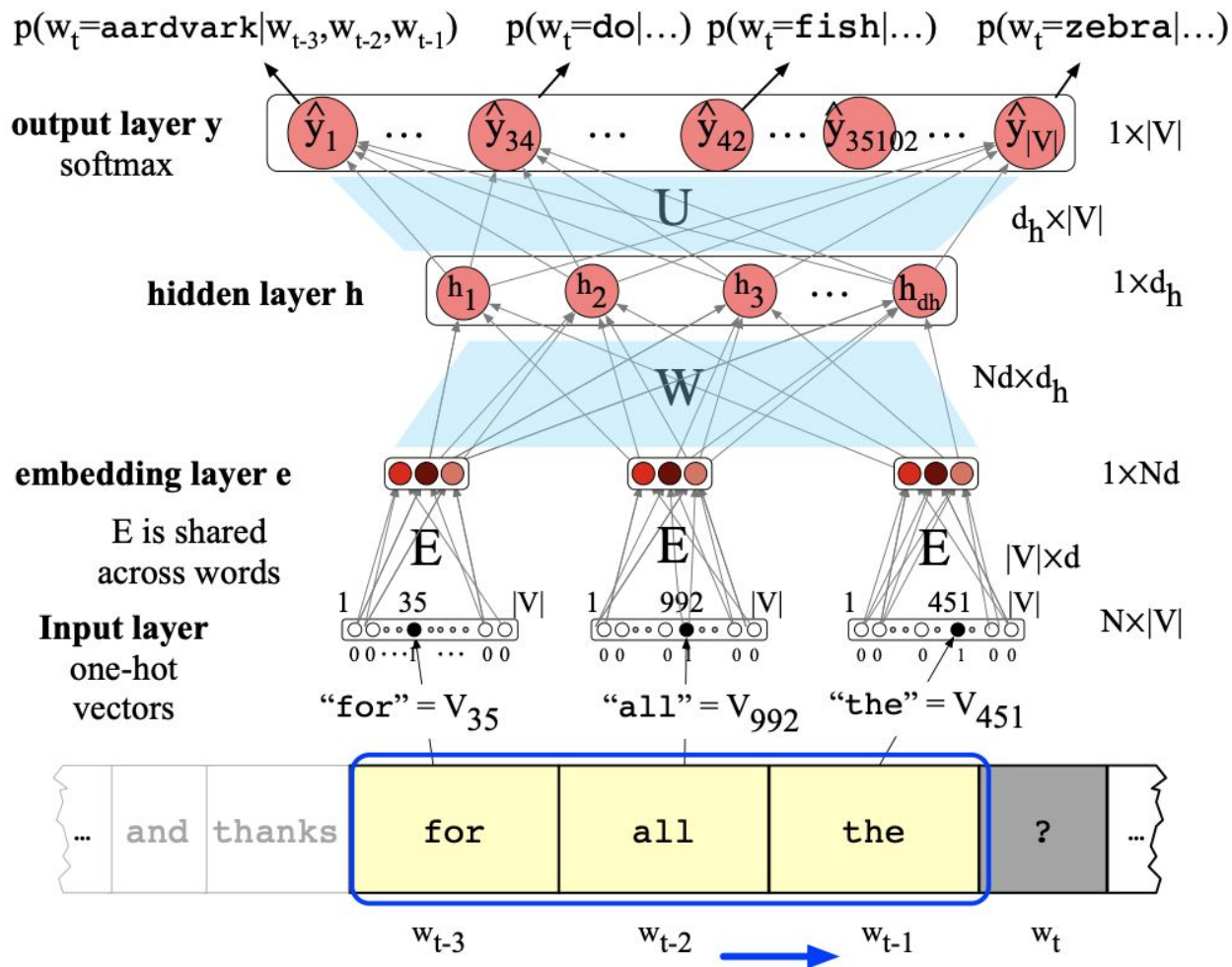
$$\mathbf{e} = [\mathbf{Ex}_{t-3}; \mathbf{Ex}_{t-2}; \mathbf{Ex}_{t-1}]$$

$$\mathbf{h} = \sigma(\mathbf{We} + \mathbf{b})$$

$$\mathbf{z} = \mathbf{Uh}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z})$$

So $\hat{y}_{42}$ is the probability of the next word w_t being $V_{42} = \text{fish}$



Why Neural LMs work better than N-gram LMs

Training data:

We've seen: I have to make sure that the cat gets fed.

Never seen: dog gets fed

Test data:

I forgot to make sure that the dog gets ____

N-gram LM can't predict "fed"!

Neural LM can use similarity of "cat" and "dog" embeddings to generalize and predict "fed" after dog

Training a FFNN

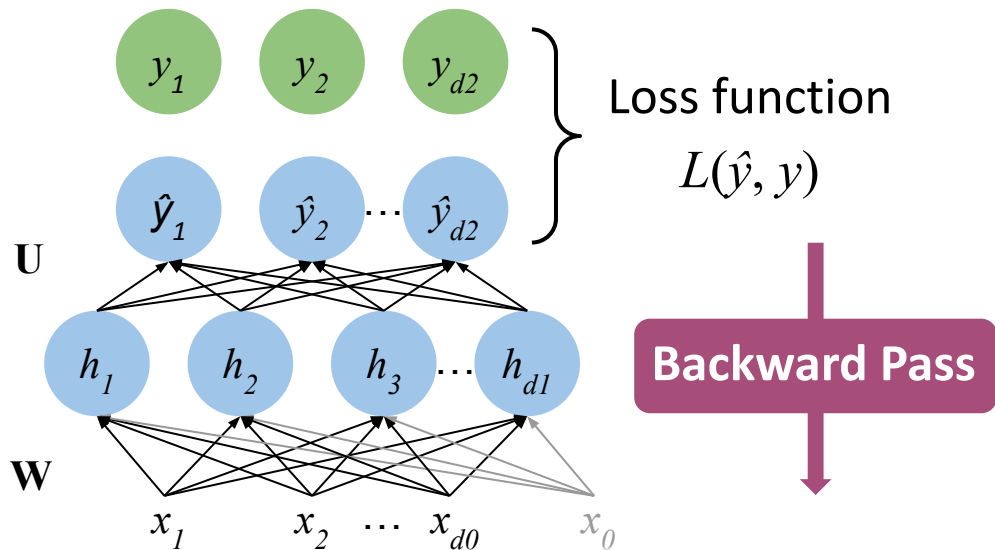
Intuition: Training a 2-layer Network

Actual Answer: y

Model Output: $\hat{y} = \text{softmax}(\mathbf{U} \cdot \mathbf{h})$

Forward Pass

Training Instance: $\mathbf{x}$



Intuition: Training a 2-layer Network

For every training tuple (x,y)

- Run **forward** computation to find our estimate $\hat{y}$
- Run **backward** computation to update weights:
 - For every output node
 - Compute loss L between true y and the estimated $\hat{y}$
 - For every weight w from hidden layer to the output layer
 - Update the weight
 - For every hidden node
 - Assess how much blame it deserves for the current answer
 - For every weight w from input layer to the hidden layer
 - Update the weight

Loss & Gradient Updates: Binary Logistic Regression

Cross entropy loss for logistic regression:

$$\begin{aligned} L_{\text{CE}}(\hat{y}, y) &= -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \\ &= -[y \log \sigma(w \cdot x + b) + (1 - y) \log(1 - \sigma(w \cdot x + b))] \end{aligned}$$

Gradient of the Loss w.r.t. the weights:

$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j$$

Loss & Gradient Updates: Binary Logistic Regression

Cross entropy loss for logistic regression:

$$\begin{aligned} L_{\text{CE}}(\hat{y}, y) &= -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \\ &= -[y \log \sigma(w \cdot x + b) + (1 - y) \log(1 - \sigma(w \cdot x + b))] \end{aligned}$$

Gradient of the Loss w.r.t. the weights:

$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y]x_j$$

Only one parameter!

Loss & Gradient Updates: Binary Logistic Regression

Cross entropy loss for logistic regression:

$$\begin{aligned} L_{\text{CE}}(\hat{y}, y) &= -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \\ &= -[y \log \sigma(w \cdot x + b) + (1 - y) \log(1 - \sigma(w \cdot x + b))] \end{aligned}$$

Gradient of the Loss w.r.t. the weights:

$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y] x_j$$

Only one parameter!

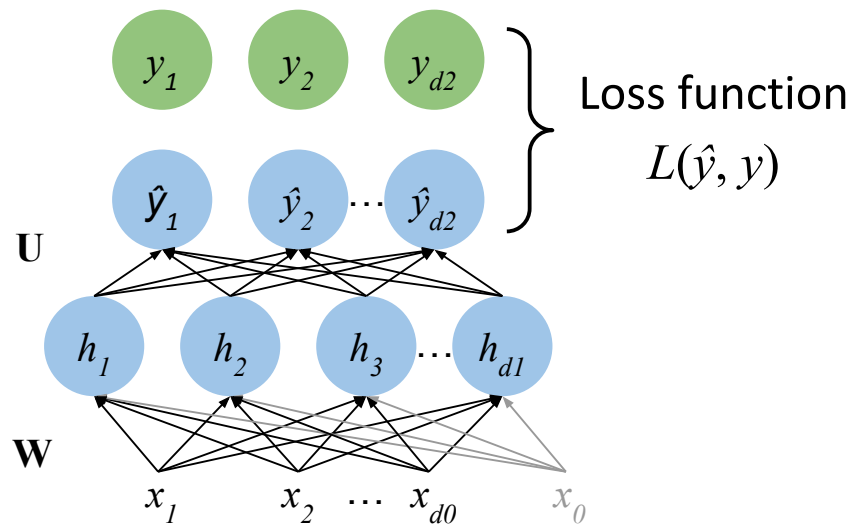
More hidden layers, more parameters (and activation functions) to consider

→ **Computation Graphs & Backward Differentiation**

Why Computation Graphs?

For training, we need the derivative of the loss with respect to each weight in every layer of the network

- But the loss is computed only at the very end of the network!



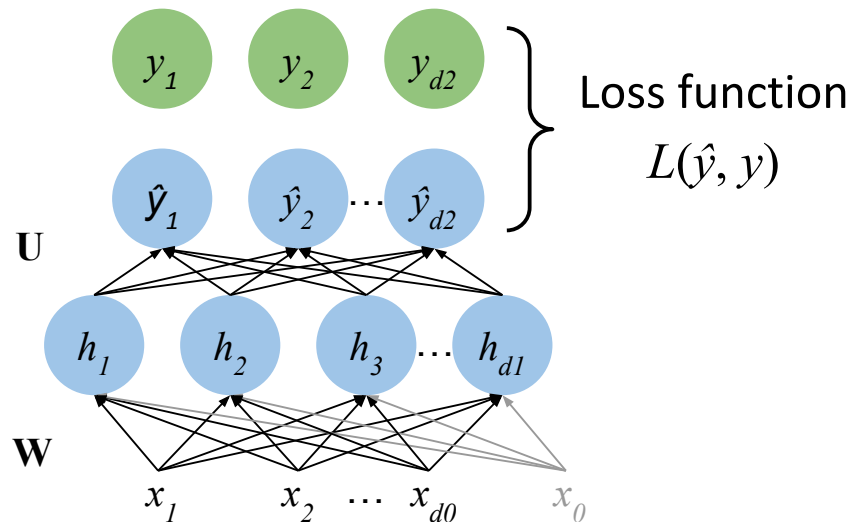
Why Computation Graphs?

For training, we need the derivative of the loss with respect to each weight in every layer of the network

- But the loss is computed only at the very end of the network!

Solution: **error backpropagation**

- **Backprop** is a special case of **backward differentiation**
 - Relies on **computation graphs**.



Computation Graph: represents the process of computing a mathematical expression

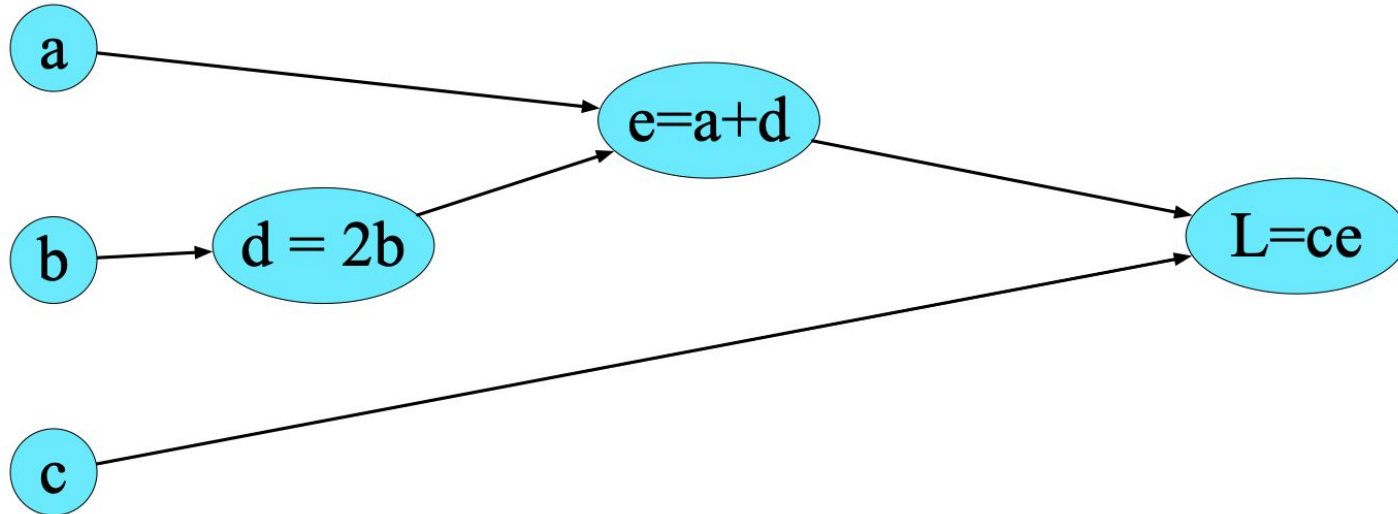
Example

$$L(a, b, c) = c(a + 2b)$$

$$d = 2 * b$$

Computations: $e = a + d$

$$L = c * e$$



Example

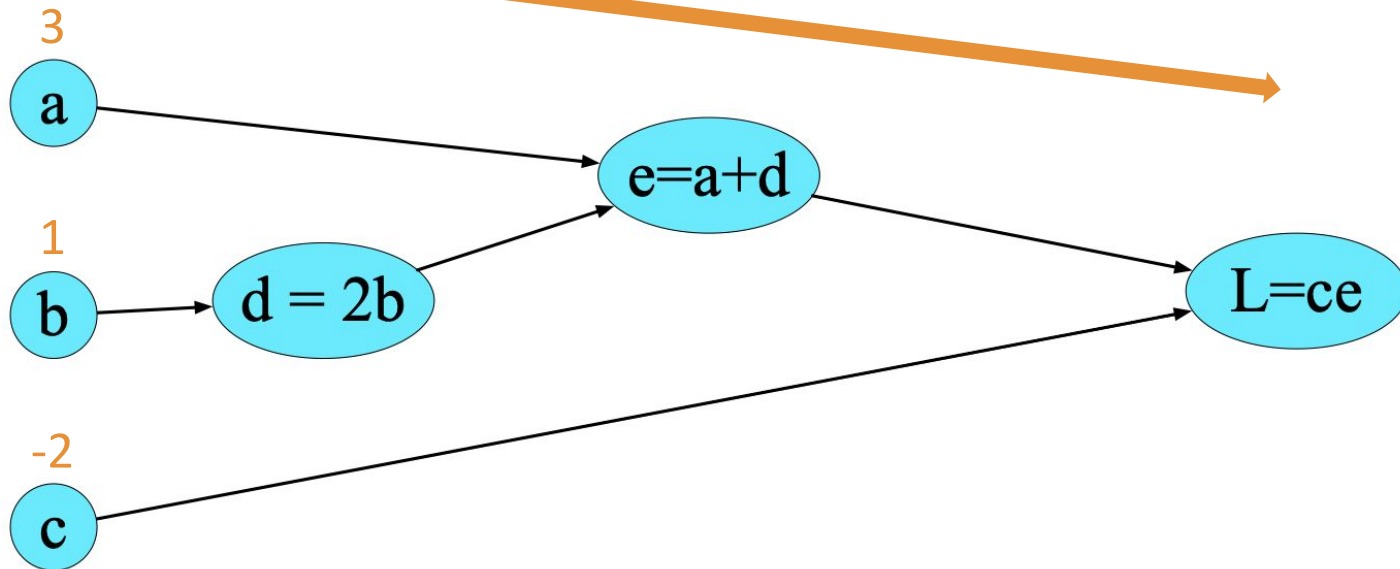
$$L(a, b, c) = c(a + 2b)$$

$$d = 2 * b$$

Computations: $e = a + d$

$$L = c * e$$

Forward Pass



Example

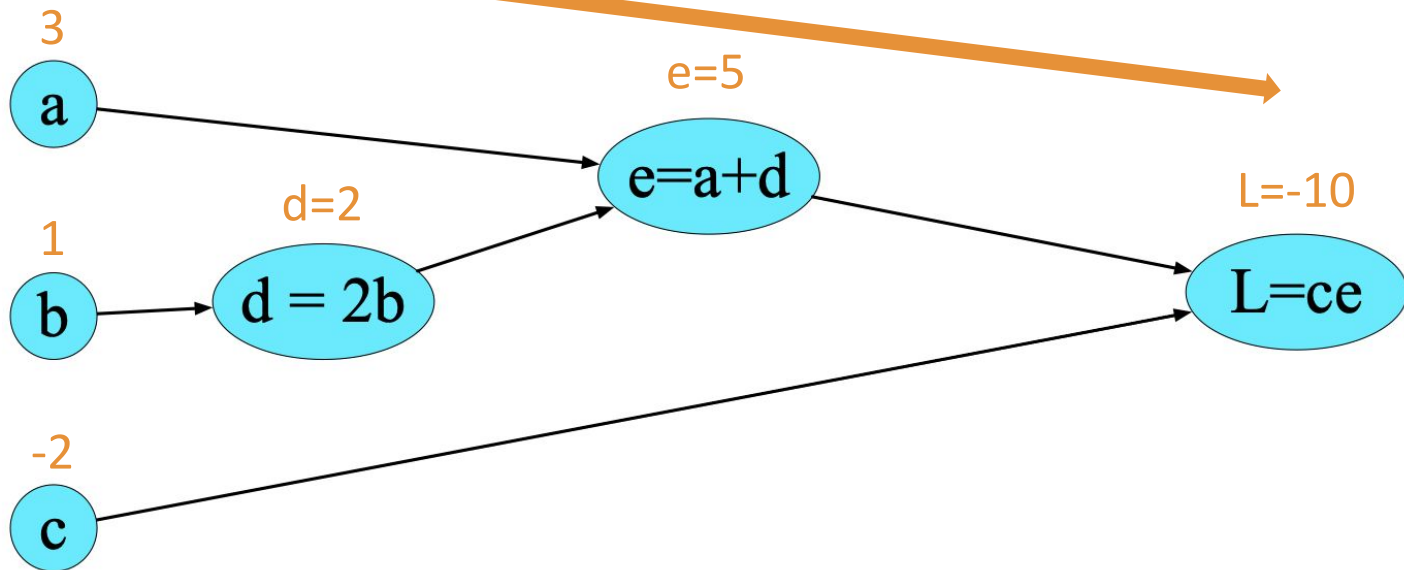
$$L(a,b,c) = c(a + 2b)$$

$$d = 2 * b$$

Computations: $e = a + d$

$$L = c * e$$

Forward Pass



Example

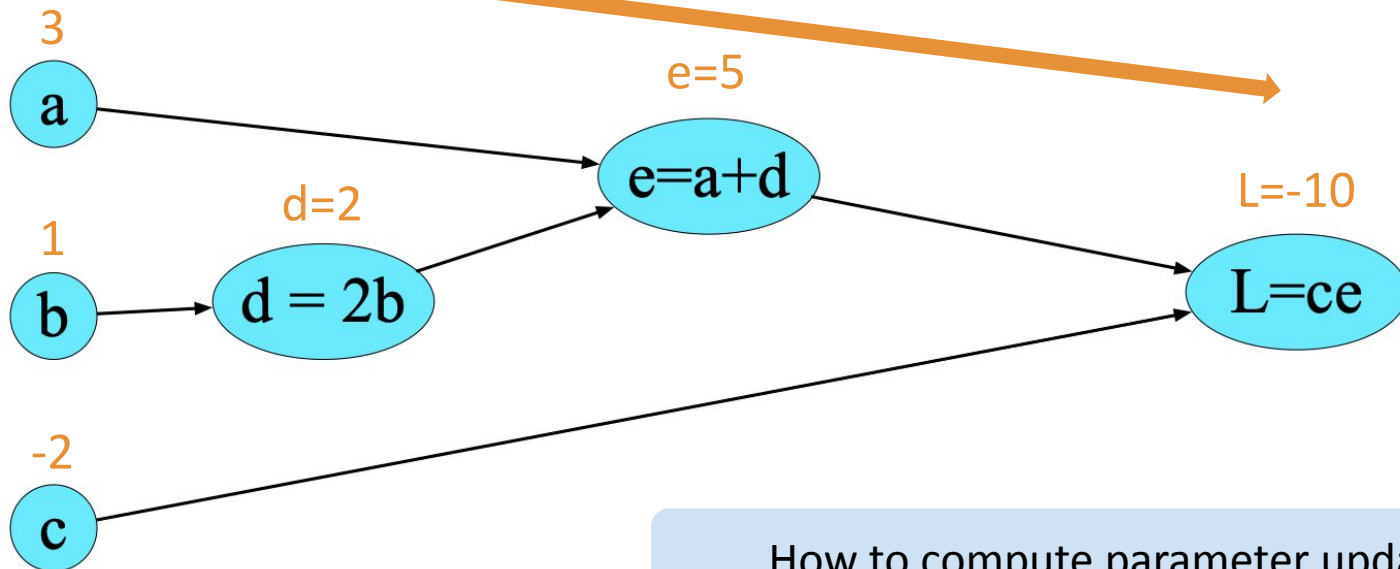
$$L(a, b, c) = c(a + 2b)$$

$$d = 2 * b$$

Computations: $e = a + d$

$$L = c * e$$

Forward Pass



How to compute parameter updates?

Backwards Differentiation in Computation Graphs

- The importance of the computation graph comes from the **backward pass**
- Used to compute the derivatives that we'll need for the weight update.

$$L(a, b, c) = c(a + 2b)$$

$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$

Tells us how a small change in a affects L

$$\frac{\partial L}{\partial a} = ?$$

$$\frac{\partial L}{\partial b} = ?$$

$$\frac{\partial L}{\partial c} = ?$$

Input layer
gradients

Backwards Differentiation in Computation Graphs

- The importance of the computation graph comes from the **backward pass**
- Used to compute the derivatives that we'll need for the weight update.

$$L(a, b, c) = c(a + 2b)$$

$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$

Tells us how a small change in a affects L

$$\frac{\partial L}{\partial a} = ?$$

$$\frac{\partial L}{\partial b} = ?$$

$$\frac{\partial L}{\partial c} = ?$$

Input layer
gradients

$$\frac{\partial L}{\partial e} = ?$$

$$\frac{\partial L}{\partial d} = ?$$

Hidden layer
gradients

The Chain Rule

Computing the derivative of a composite function:

$$f(x) = u(v(x)) \qquad \frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$$

$$f(x) = u(v(w(x))) \qquad \frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dw} \cdot \frac{dw}{dx}$$

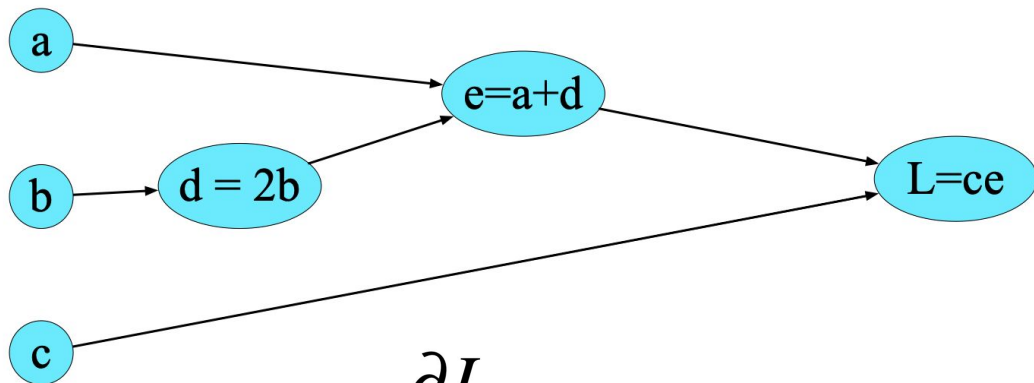
Example

$$L(a, b, c) = c(a + 2b)$$

$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$



3 derivatives
for input
variables:

$$\frac{\partial L}{\partial c} = e$$

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b}$$

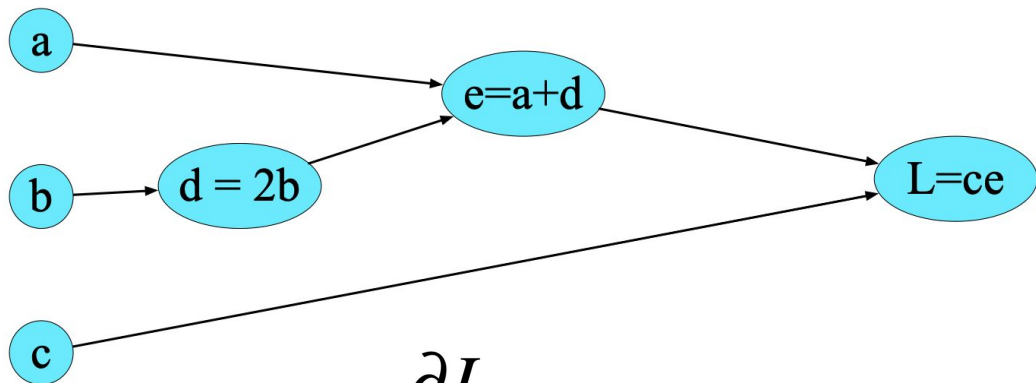
Example

$$L(a, b, c) = c(a + 2b)$$

$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$



3 derivatives
for input
variables:

$$\frac{\partial L}{\partial c} =$$

$$\frac{\partial L}{\partial a} =$$

$$\frac{\partial L}{\partial b} =$$

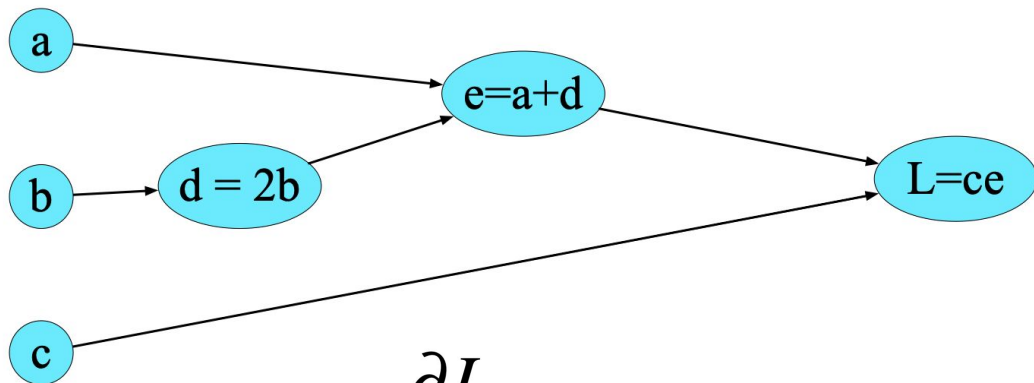
Example

$$L(a, b, c) = c(a + 2b)$$

$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$



3 derivatives
for input
variables:

$$\frac{\partial L}{\partial c} = e$$

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b}$$

$$L = ce : \quad \frac{\partial L}{\partial e}$$

$$e = a + d : \quad \frac{\partial e}{\partial a} \quad \frac{\partial e}{\partial d} =$$

$$d = 2b : \quad \frac{\partial d}{\partial b}$$

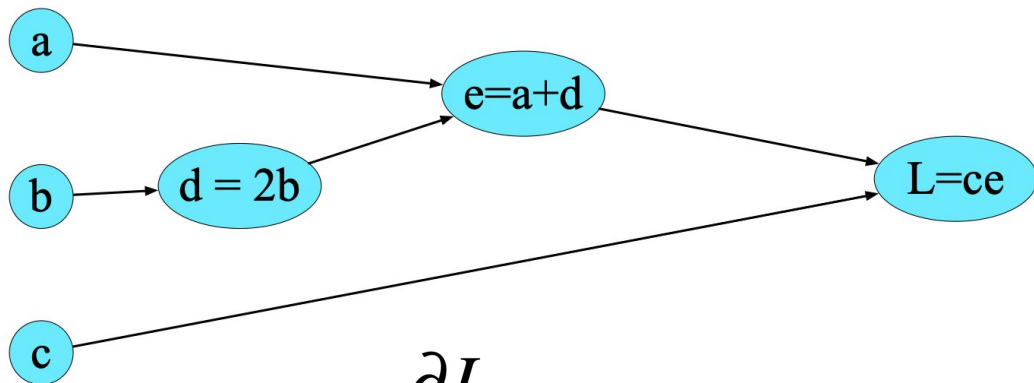
Example

$$L(a, b, c) = c(a + 2b)$$

$$d = 2 * b$$

$$e = a + d$$

$$L = c * e$$



3 derivatives
for input
variables:

$$\frac{\partial L}{\partial c} = e$$

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b}$$

$$L = ce : \quad \frac{\partial L}{\partial e} \quad \frac{\partial L}{\partial c} = e$$

$$e = a + d : \quad \frac{\partial e}{\partial a} \quad \frac{\partial e}{\partial d} = 1$$

$$d = 2b : \quad \frac{\partial d}{\partial b}$$

Example

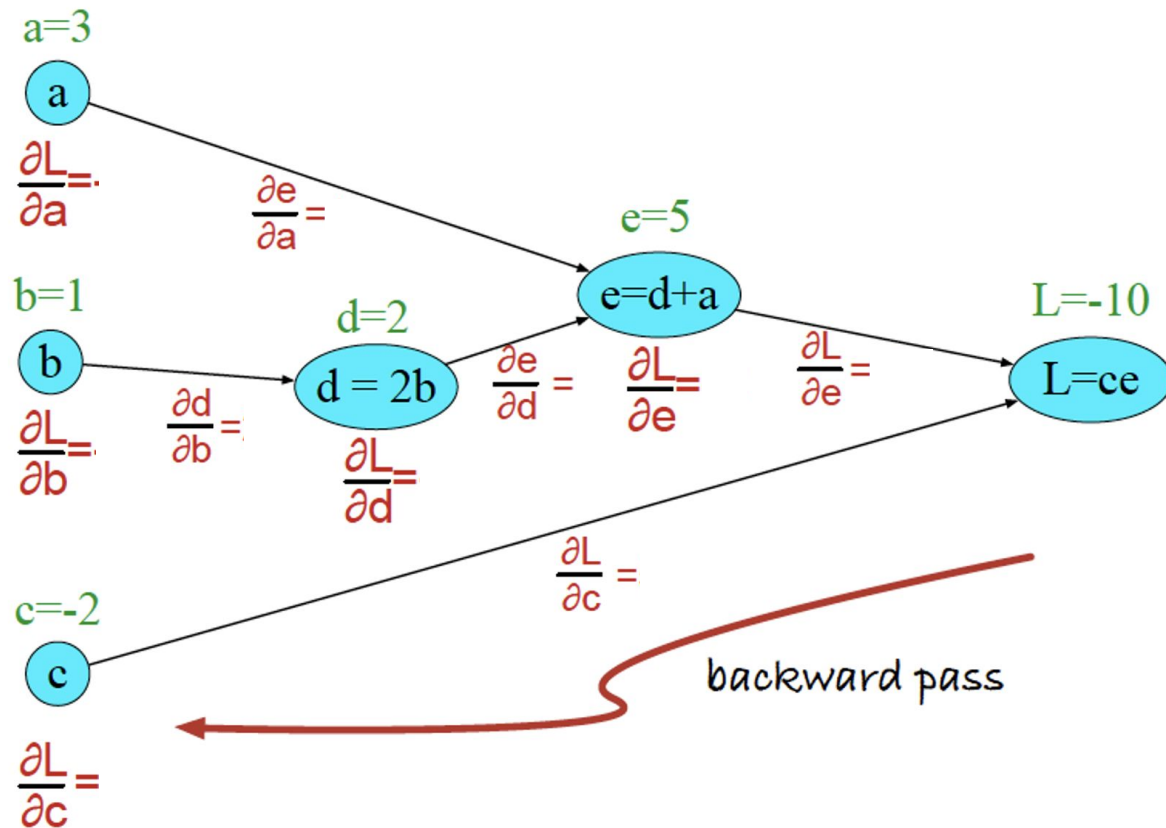
$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b}$$

$$L = ce : \quad \frac{\partial L}{\partial e} = c, \quad \frac{\partial L}{\partial c} = e$$

$$e = a + d : \quad \frac{\partial e}{\partial a} = 1, \quad \frac{\partial e}{\partial d} = 1$$

$$d = 2b : \quad \frac{\partial d}{\partial b} = 2$$



Example

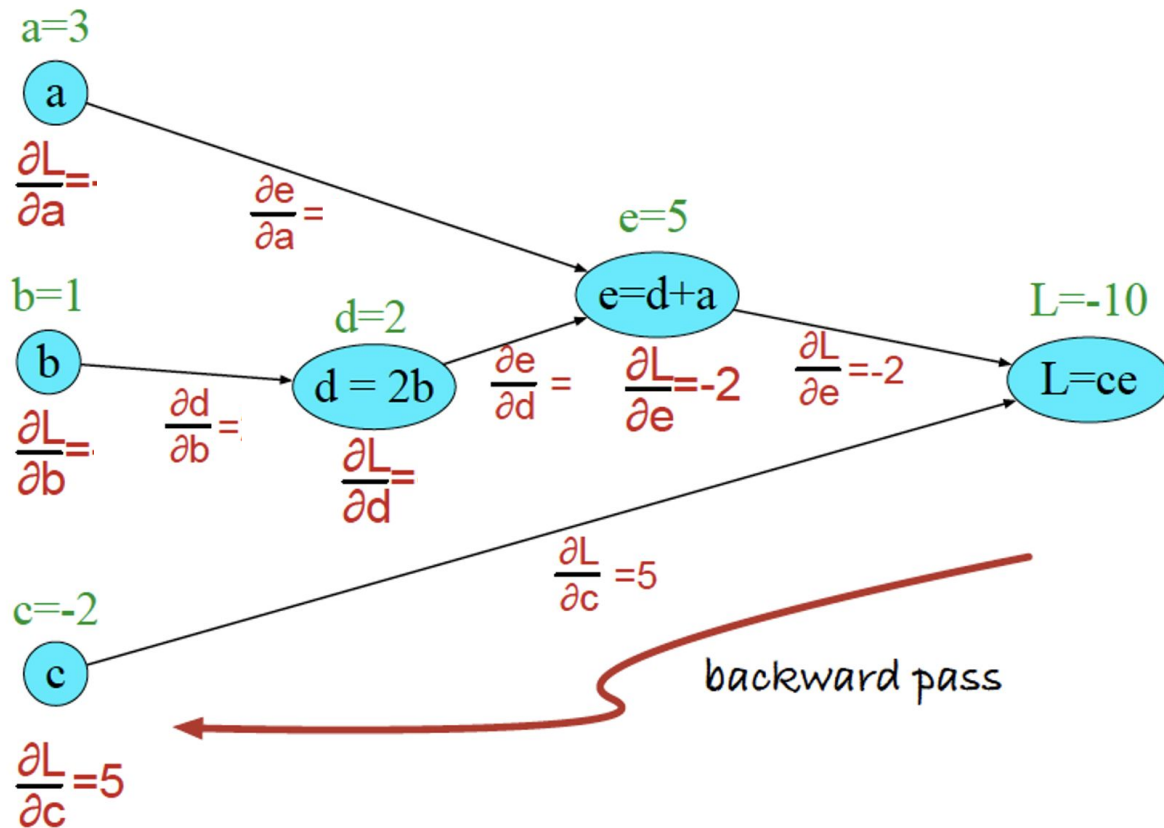
$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b}$$

$$L = ce : \quad \frac{\partial L}{\partial e} = c, \frac{\partial L}{\partial c} = e$$

$$e = a + d : \quad \frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial d} = 1$$

$$d = 2b : \quad \frac{\partial d}{\partial b} = 2$$



Example

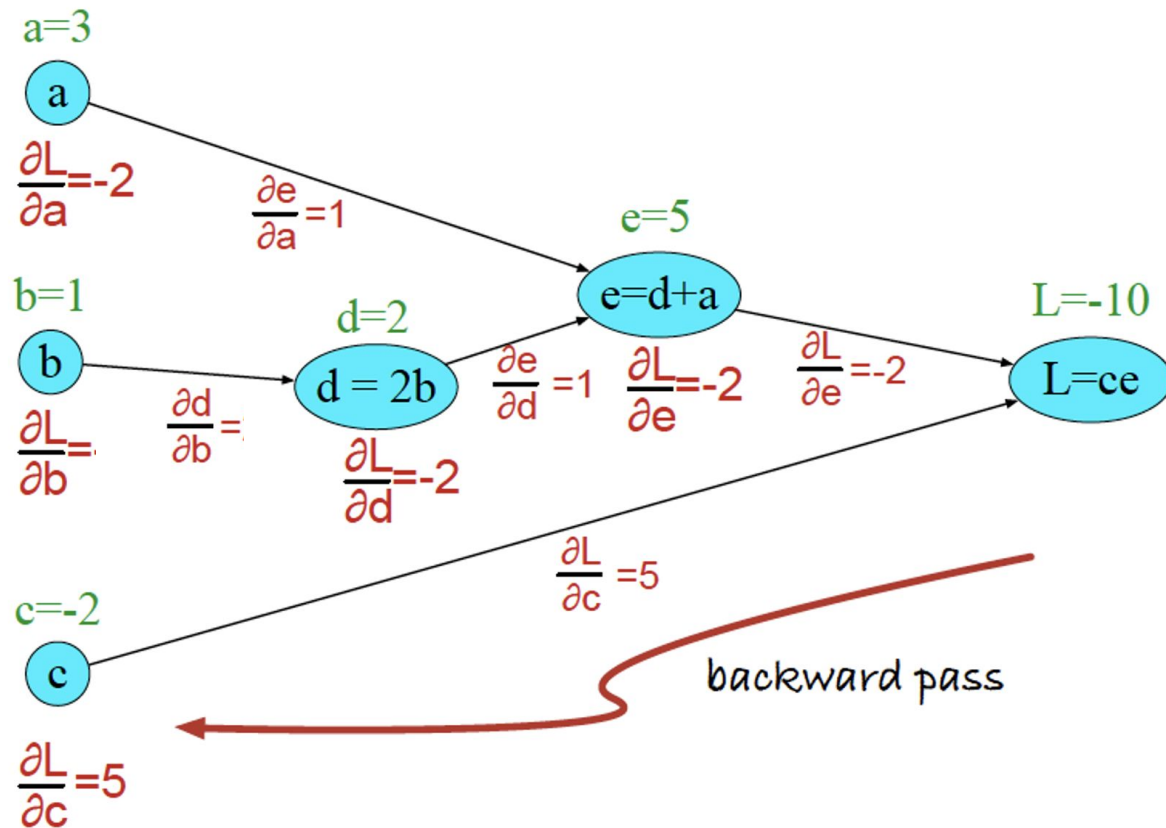
$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b}$$

$$L = ce : \quad \frac{\partial L}{\partial e} = c, \frac{\partial L}{\partial c} = e$$

$$e = a + d : \quad \frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial d} = 1$$

$$d = 2b : \quad \frac{\partial d}{\partial b} = 2$$



Example

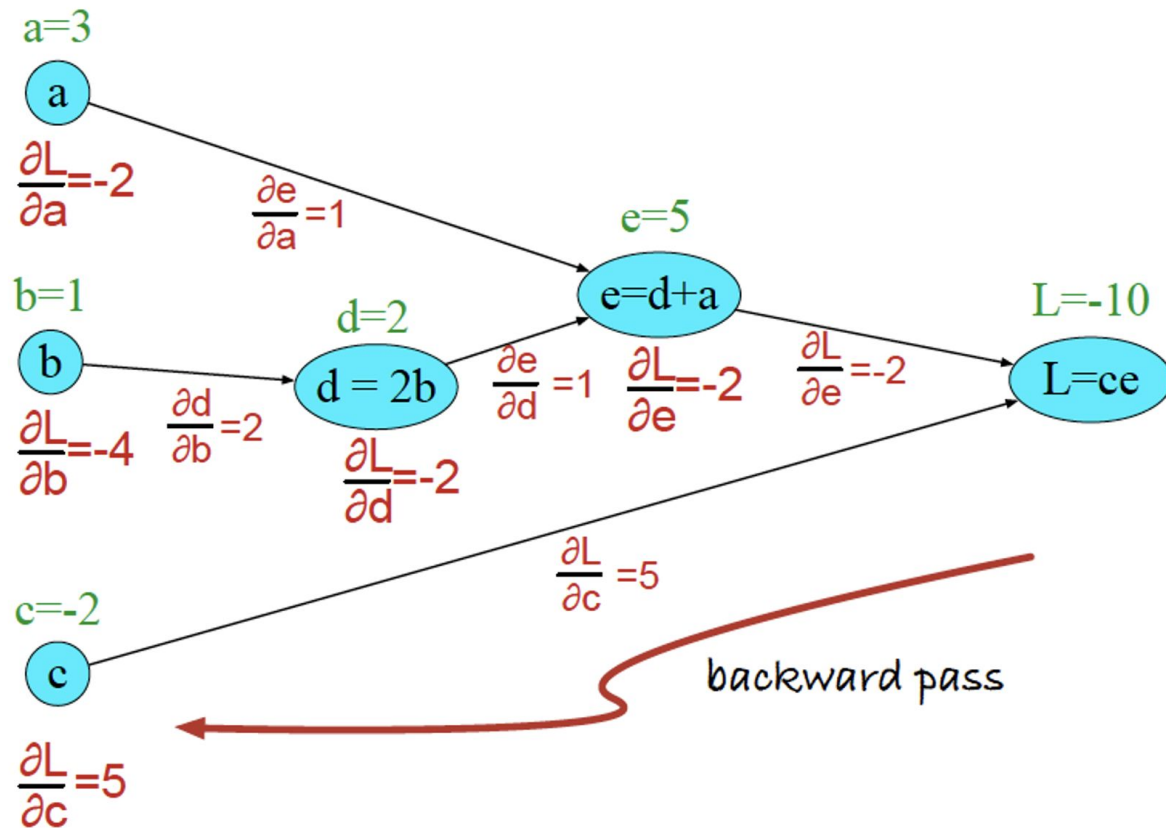
$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial a}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial e} \frac{\partial e}{\partial d} \frac{\partial d}{\partial b}$$

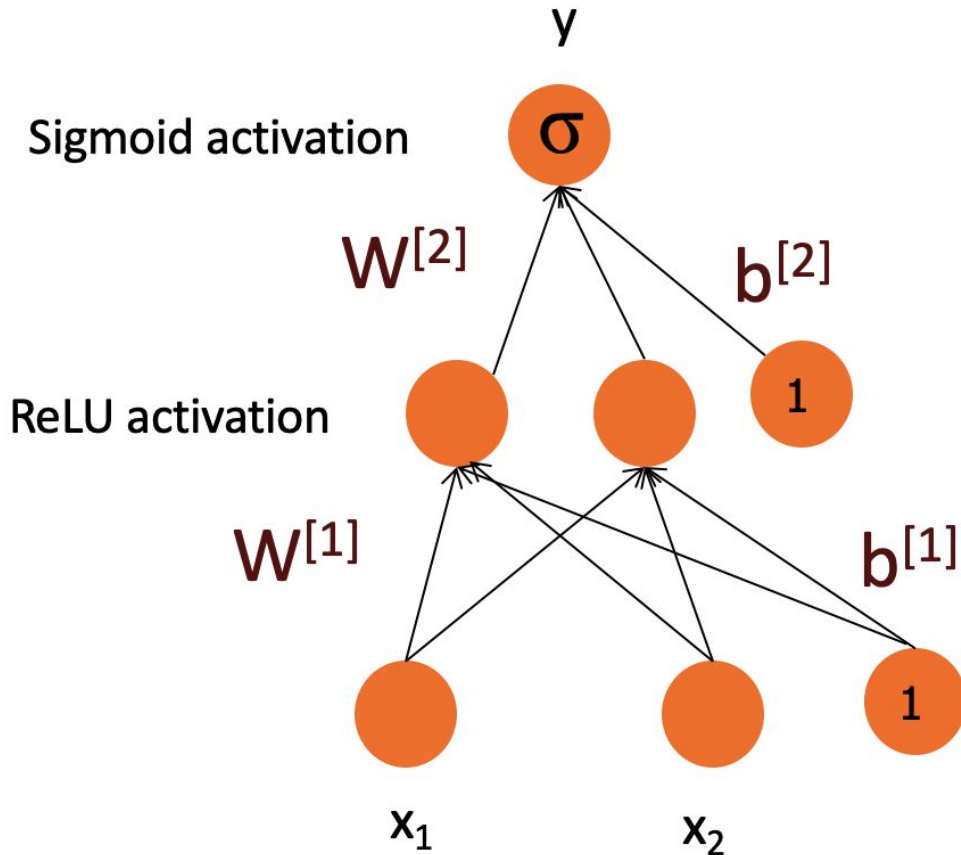
$$L = ce : \quad \frac{\partial L}{\partial e} = c, \frac{\partial L}{\partial c} = e$$

$$e = a + d : \quad \frac{\partial e}{\partial a} = 1, \frac{\partial e}{\partial d} = 1$$

$$d = 2b : \quad \frac{\partial d}{\partial b} = 2$$



Backward differentiation on a two layer network



$$\begin{aligned}z^{[1]} &= W^{[1]} \mathbf{x} + b^{[1]} \\a^{[1]} &= \text{ReLU}(z^{[1]}) \\z^{[2]} &= W^{[2]} a^{[1]} + b^{[2]} \\a^{[2]} &= \sigma(z^{[2]}) \\\hat{y} &= a^{[2]}\end{aligned}$$

Backward differentiation on a 2-layer network

$$z^{[1]} = W^{[1]}\mathbf{x} + b^{[1]}$$

$$a^{[1]} = \text{ReLU}(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

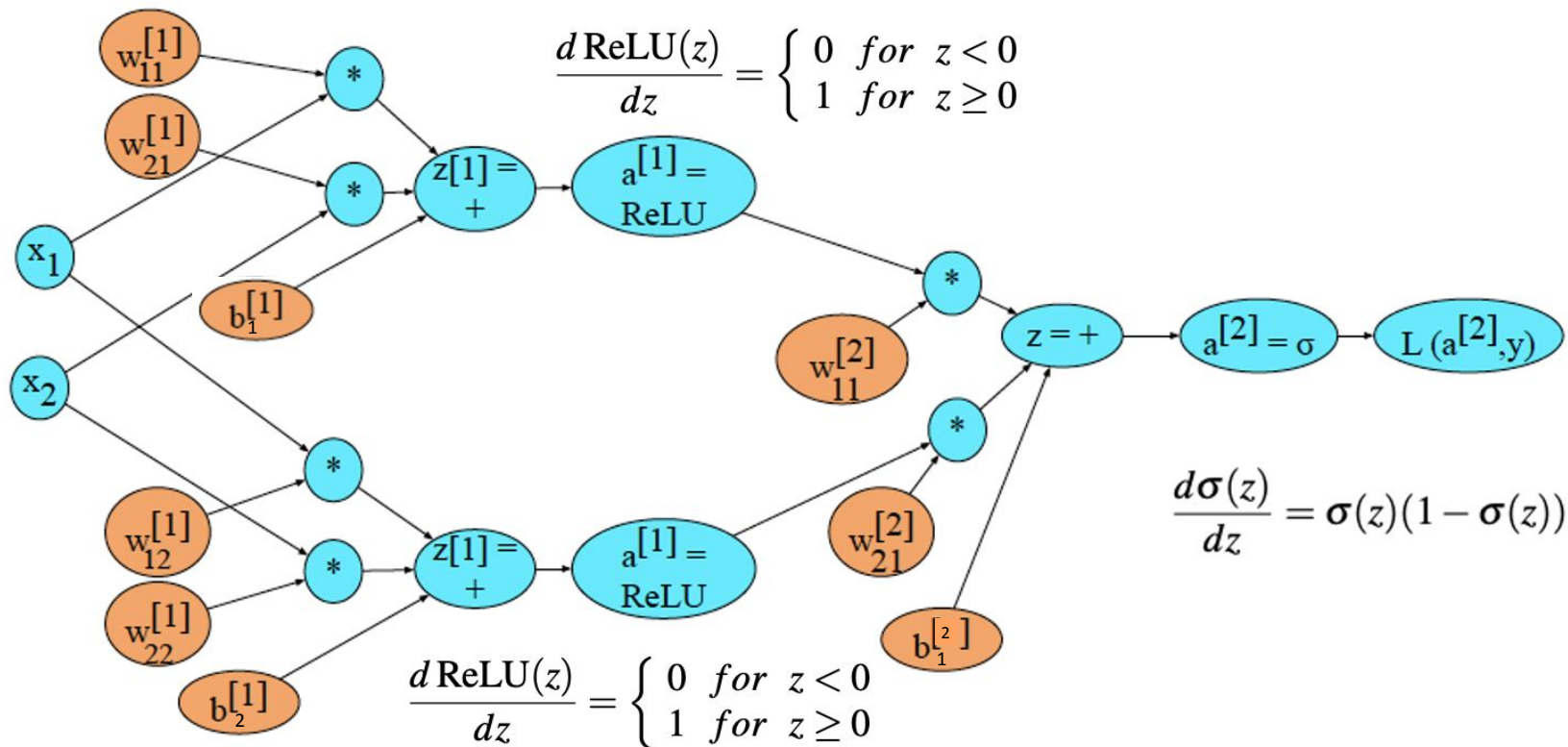
$$a^{[2]} = \sigma(z^{[2]})$$

$$\hat{y} = a^{[2]}$$

$$\frac{d\text{ReLU}(z)}{dz} = \begin{cases} 0 & \text{for } z < 0 \\ 1 & \text{for } z \geq 0 \end{cases}$$

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z))$$

Backward differentiation on a 2-layer network



Starting off the backward pass: $\frac{\partial L}{\partial \mathbf{z}}$

(I'll write a for $a^{[2]}$ and z for $z^{[2]}$)

$$L(\hat{y}, y) = -(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y}))$$

$$L(a, y) = -(y \log a + (1 - y) \log(1 - a))$$

$$\frac{\partial L}{\partial \mathbf{z}} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial \mathbf{z}}$$

$$\begin{aligned} \frac{\partial L}{\partial a} &= - \left(\left(y \frac{\partial \log(a)}{\partial a} \right) + (1 - y) \frac{\partial \log(1 - a)}{\partial a} \right) \\ &= - \left(\left(y \frac{1}{a} \right) + (1 - y) \frac{1}{1 - a} (-1) \right) = - \left(\frac{y}{a} + \frac{y - 1}{1 - a} \right) \end{aligned}$$

$$\frac{\partial a}{\partial \mathbf{z}} = a(1 - a)$$

$$\frac{\partial L}{\partial \mathbf{z}} = - \left(\frac{y}{a} + \frac{y - 1}{1 - a} \right) a(1 - a) = a - y$$

$$\mathbf{z}^{[1]} = \mathbf{W}^{[1]} \mathbf{x} + \mathbf{b}^{[1]}$$

$$\mathbf{a}^{[1]} = \text{ReLU}(\mathbf{z}^{[1]})$$

$$\mathbf{z}^{[2]} = \mathbf{W}^{[2]} \mathbf{a}^{[1]} + \mathbf{b}^{[2]}$$

$$\mathbf{a}^{[2]} = \sigma(\mathbf{z}^{[2]})$$

$$\hat{y} = \mathbf{a}^{[2]}$$

Summary

- For training, we need the derivative of the loss with respect to weights in early layers of the network
 - But loss is computed only at the very end of the network!
- Solution: **backward differentiation**
- Given a computation graph and the derivatives of all the functions in it we can automatically compute the derivative of the loss with respect to these early weights.

TODOs:

- **Homework 1 Due: 9/30**
- **Project Proposal Due: 10/02**

Next Class: RNNs